FPT SCHOOL OF BUSINESS & TECHNOLOGY

# AI - MACHINE LEARNING

PhD. Truc Thi Kim, Nguyen

093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

# ML Strategies

FSB

# ML Strategy 1



- Why ML Strategy
- Orthogonalization
- Single number evaluation metric
- Satisfying and Optimizing metric
- Train/dev/test distributions
- Size of the dev and test sets
- When to change dev/test sets and metrics
- Why human-level performance?
- Avoidable bias
- Understanding human-level performance
- Surpassing human-level performance
- Improving your model performance

FSB

# Why ML Strategy



- You have a lot of ideas for how to improve the accuracy of your deep learning system:
  - Collect more data.
  - Collect more diverse training set.
  - Train algorithm longer with gradient descent.
  - Try different optimization algorithm (e.g. Adam).
  - Try bigger network.
  - Try smaller network.
  - Try dropout.
  - Add L2 regularization.
  - Change network architecture (activation functions, # of hidden units, etc.)
- This course will give you some strategies to help analyze your problem to go in a direction that will help you get better results.

FSB

# Improving your model performance



- The two fundamental assumptions of supervised learning:
  - You can fit the training set pretty well. This is roughly saying that you can achieve low **avoidable bias**.
  - The training set performance generalizes pretty well to the dev/test set. This is roughly saying that **variance** is not too bad.
- To improve your deep learning supervised system follow these guidelines:
  - Look at the difference between human level error and the training error - **avoidable bias**.
  - Look at the difference between the dev/test set and training set error - **Variance**.
  - If **avoidable bias** is large you have these options:
    - Train bigger model.
    - Train longer/better optimization algorithm (like Momentum, RMSprop, Adam).
    - Find better NN architecture/hyperparameters search.
  - If **variance** is large you have these options:
    - Get more training data.
    - Regularization (L2, Dropout, data augumentation).
    - Find better NN architecture/hyperparameters search.

FSB

# ML Strategy 2



- Carrying out error analysis
- Cleaning up incorrectly labeled data
- Build your first system quickly, then iterate
- Training and testing on different distributions
- Bias and Variance with mismatched data distributions
- Addressing data mismatch
- Transfer learning
- Multi-task learning
- What is end-to-end deep learning?
- Whether to use end-to-end deep learning

FSB

# Carrying out error analysis



Error analysis - process of manually examining mistakes that your algorithm is making. It can give you insights into what to do next. E.g.:

- In the cat classification example, if you have 10% error on your dev set and you want to decrease the error.
- You discovered that some of the mislabeled data are dog pictures that look like cats. Should you try to make your cat classifier do better on dogs (this could take some weeks)?
- Error analysis approach:
  - Get 100 mislabeled dev set examples at random.
  - Count up how many are dogs.
  - if 5 of 100 are dogs then training your classifier to do better on dogs will decrease your error up to 9.5% (called ceiling), which can be too little.
  - if 50 of 100 are dogs then you could decrease your error up to 5%, which is reasonable and you should work on that.
- Based on the last example, error analysis helps you to analyze the error before taking an action that could take lot of time with no need.

# Training and testing on different distributions



- A lot of teams are working with deep learning applications that have training sets that are different from the dev/test sets due to the hunger of deep learning to data.
- There are some strategies to follow up when training set distribution differs from dev/test sets distribution.
  - Option one (not recommended): shuffle all the data together and extract randomly training and dev/test sets.
    - Advantages: all the sets now come from the same distribution.
    - Disadvantages: the other (real world) distribution that was in the dev/test sets will occur less in the new dev/test sets and that might be not what you want to achieve.
  - Option two: take some of the dev/test set examples and add them to the training set.
    - Advantages: the distribution you care about is your target now.
    - Disadvantage: the distributions in training and dev/test sets are now different. But you will get a better performance over a long time.

FSB

# Addressing data mismatch



- There aren't completely systematic solutions to this, but there some things you could try.

1. Carry out manual error analysis to try to understand the difference between training and dev/test sets.
2. Make training data more similar, or collect more data similar to dev/test sets.

- If your goal is to make the training data more similar to your dev set one of the techniques you can use **Artificial data synthesis** that can help you make more training data.
  - Combine some of your training data with something that can convert it to the dev/test set distribution.
    - Examples:
      - Combine normal audio with car noise to get audio with car noise example.
      - Generate cars using 3D graphics in a car classification example.
  - Be cautious and bear in mind whether or not you might be accidentally simulating data only from a tiny subset of the space of all possible examples because your NN might overfit these generated data (like particular car noise or a particular design of 3D graphics cars).

# Transfer learning

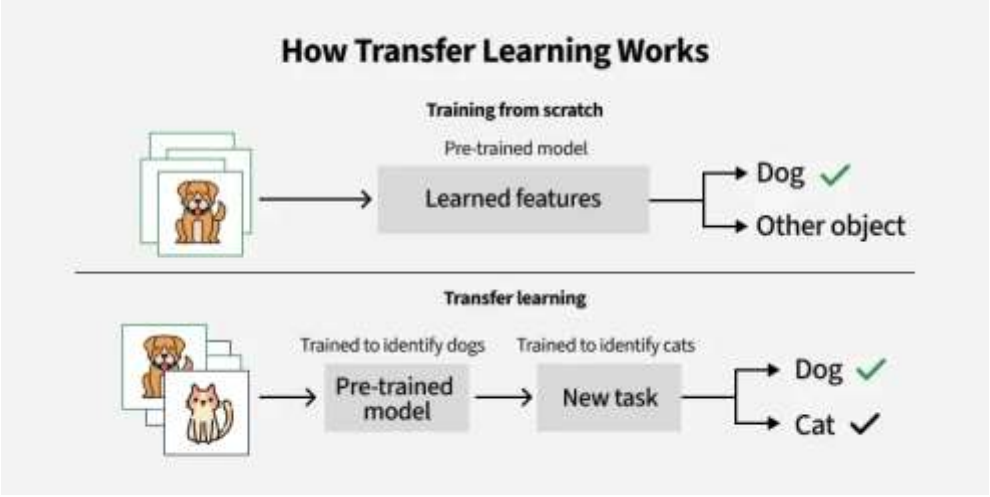


- Apply the knowledge you took in a task A and apply it in another task B.
- For example, you have trained a cat classifier with a lot of data, you can use the part of the trained NN it to solve x-ray classification problem.
- To do transfer learning, delete the last layer of NN and it's weights and:
  - Option 1: if you have a small data set - keep all the other weights as a fixed weights. Add a new last layer(-s) and initialize the new layer weights and feed the new data to the NN and learn the new weights.
  - Option 2: if you have enough data you can retrain all the weights.
- Option 1 and 2 are called **fine-tuning** and training on task A called **pretraining**.
- When transfer learning make sense:
  - Task A and B have the same input X (e.g. image, audio).
  - You have a lot of data for the task A you are transferring from and relatively less data for the task B your transferring to.
  - Low level features from task A could be helpful for learning task B.

# What is Transfer Learning?



- **Transfer learning** is a machine learning technique where a model trained on one task is repurposed as the foundation for a second task. This approach is beneficial when the second task is related to the first or when data for the second task is limited.




FSB

# What is Transfer Learning?



- This approach is beneficial when the second task is related to the first or when data for the second task is limited.
- Leveraging learned features from the initial task, the model can adapt more efficiently to the new task, accelerating learning and improving performance.
- Transfer learning also reduces the risk of overfitting, as the model already incorporates generalizable features useful for the second task.

FSB

# Why is Transfer Learning Important?



Transfer learning offer solutions to key challenges like:

- **Limited Data:** Acquiring extensive labeled data is often challenging and costly. Transfer learning enables us to use pre-trained models, reducing the dependency on large datasets.
- **Enhanced Performance:** Starting with a pre-trained model, which has already learned from substantial data, allows for faster and more accurate results on new tasks—ideal for applications needing high accuracy and efficiency.
- **Time and Cost Efficiency:** Transfer learning shortens training time and conserves resources by utilizing existing models, eliminating the need for training from scratch.
- **Adaptability:** Models trained on one task can be fine-tuned for related tasks, making transfer learning versatile for various applications, from image recognition to natural language processing.

FSB

# How Does Transfer Learning Work?



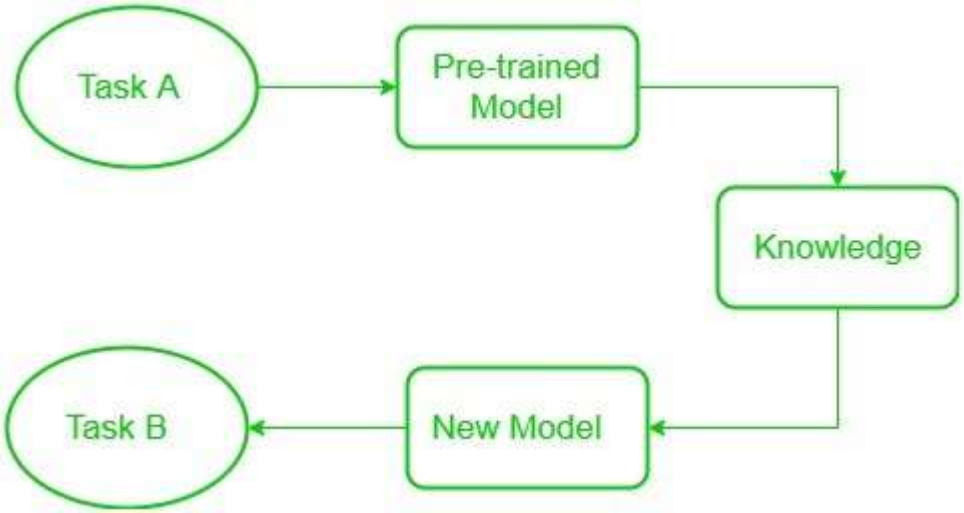
Transfer learning involves a structured process to leverage existing knowledge from a pre-trained model for new tasks:

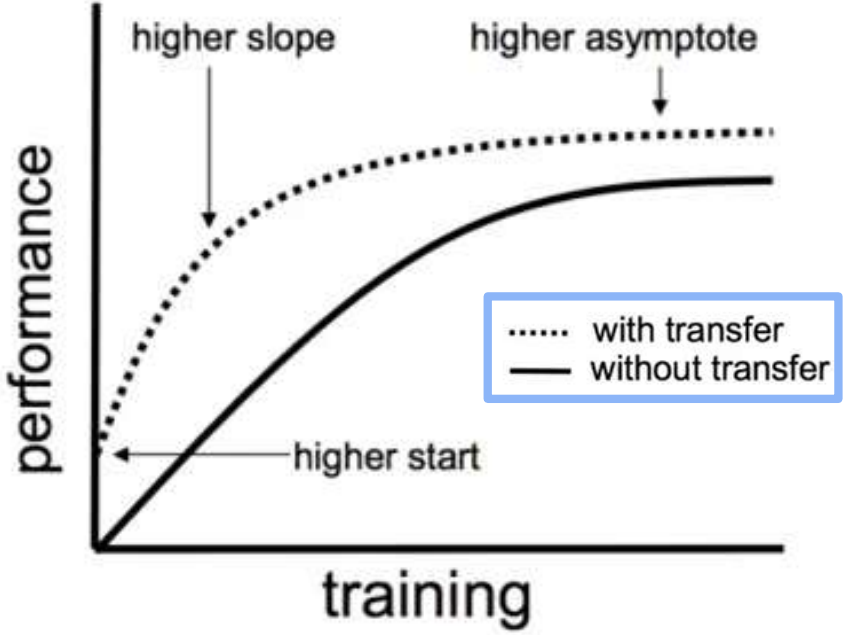
- **Pre-trained Model**: Start with a model already trained on a large dataset for a specific task. This pre-trained model has learned general features and patterns that are relevant across related tasks.
- **Base Model**: This pre-trained model, known as the base model, includes layers that have processed data to learn hierarchical representations, capturing low-level to complex features.
- **Transfer Layers**: Identify layers within the base model that hold generic information applicable to both the original and new tasks. These layers, often near the top of the network, capture broad, reusable features.
- **Fine-tuning**: Fine-tune these selected layers with data from the new task. This process helps retain the pre-trained knowledge while adjusting parameters to meet the specific requirements of the new task, improving accuracy and adaptability.

# How Does Transfer Learning Work?



- **Low-level features** learned for task A should be beneficial for learning of model for task B.



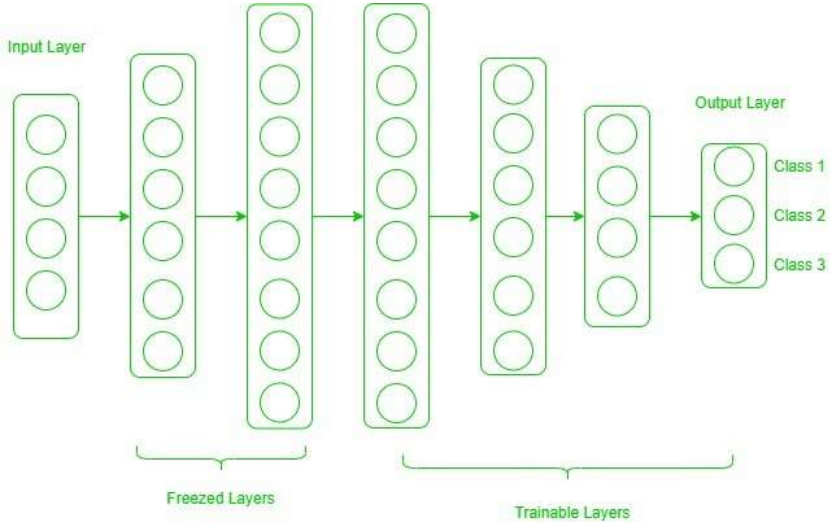



# Frozen vs. Trainable Layers in Transfer Learning



In Transfer Learning, two main components help in adapting models effectively:

- **Frozen Layers:** These layers from a pre-trained model remain unchanged during fine-tuning. They retain general features learned from the original task, extracting universal patterns from input data.
- **Trainable Layers:** These layers are adjusted during fine-tuning to learn task-specific features from the new dataset, allowing the model to meet the new task's unique requirements.
- 




FSB

# How to Decide Which Layers to Freeze or Train?



The extent to which you freeze or fine-tune layers depends on the similarity and size of your target dataset:

- **Small, Similar Dataset**: For smaller datasets that resemble the original dataset, you freeze most layers and only fine-tune the last one or two layers to prevent overfitting.
- **Large, Similar Dataset**: With large, similar datasets, you can unfreeze more layers, allowing the model to adapt while retaining learned features from the base model.
- **Small, Different Dataset**: For smaller, dissimilar datasets, fine-tuning layers closer to the input layer helps the model learn task-specific features from scratch.
- **Large, Different Dataset**: In this case, fine-tuning the entire model helps the model adapt to the new task while leveraging the broad knowledge from the pre-trained model.

# Leveraging Transfer Learning with MobileNetV2 for MNIST Classification



- We'll explore transfer learning by fine-tuning and freezing a MobileNetV2 model, pre-trained on ImageNet, for classifying MNIST digits.

- https://colab.research.google.com/drive/1SR__A3naMUHIQ9EanNPFOSfJLwEkoRFn?usp=sharing

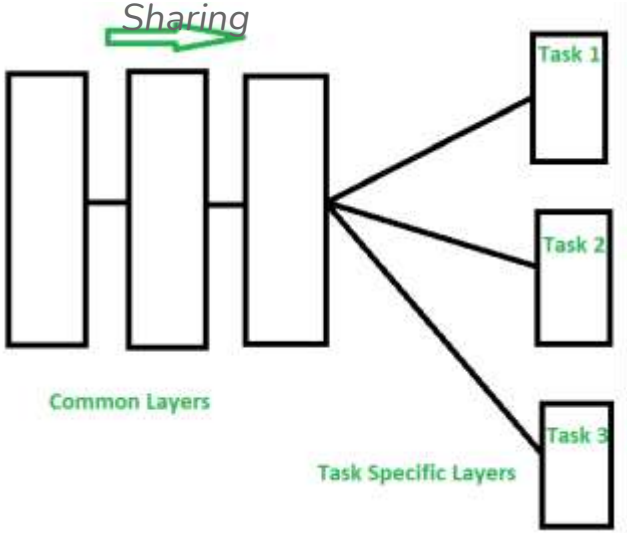
- **HOMEWORK**: do the same task but using other pretrained models of ImageNet then compare these fine-tuning models to choose the best model for this task.

# Multi-task learning

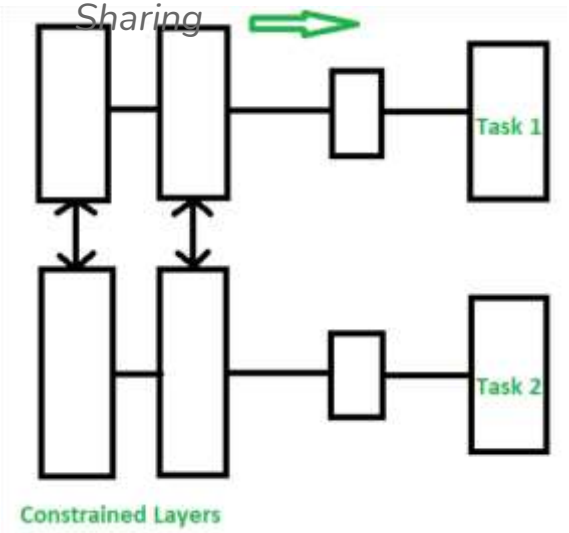


- Whereas in transfer learning, you have a sequential process where you learn from task A and then transfer that to task B. In multi-task learning, you start off simultaneously, trying to have one neural network do several things at the same time. And then each of these tasks helps hopefully all of the other tasks.

*Hard Parameter Sharing*




*Soft Parameter Sharing*



# Multi-task learning



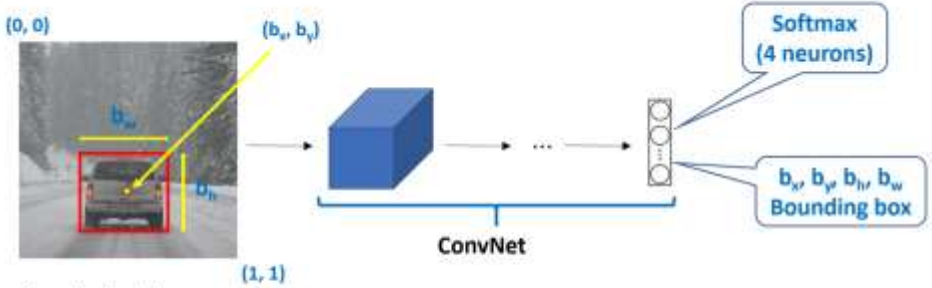
Example:

- You want to build an object recognition system that detects pedestrians, cars, stop signs, and traffic lights (image has multiple labels).
- Then Y shape will be (4,m) because we have 4 classes and each one is a binary one.
- Then: cost function

$$J = \frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{4}\left[-y_j^{(i)}\log(\hat{y}_j^{(i)}) - (1-y_j^{(i)})\log(1-\hat{y}_j^{(i)})\right]$$

- $m$ = Number of training samples
- $y_j^{(i)}$ = True label (ground truth) for the $j$-th class in the $i$-th sample (0 or 1)
- $\hat{y}_j^{(i)}$ = Predicted probability for the $j$-th class in the $i$-th sample
- $L(y_j^{(i)}, \hat{y}_j^{(i)})$ = Binary cross-entropy loss for the $j$-th class in $i$-th sample

Classification with localization: Single object detection




1. Pedestrian
2. Car
3. Motorcycle
4. Background

What is target the labels and compute loss of ?

Eg.

$b_x = 0.5$
$b_y = 0.7$
$b_h = 0.1$
$b_w = 0.4$

# Multi-task learning



- Multi-task learning makes sense:
  - Training on a set of tasks that could benefit from having shared lower-level features.
  - Usually, amount of data you have for each task is quite similar.
  - Can train a big enough network to do well on all the tasks.
- If you can train a big enough NN, the performance of the multi-task learning compared to splitting the tasks is better.
- Today transfer learning is used more often than multi-task learning.

FSB

# What is end-to-end deep learning?



- Some systems have multiple stages to implement. An end-to-end deep learning system implements all these stages with a single NN.
  - Speech recognition system:

```
Audio ---> Features --> Phonemes --> Words --> Transcript    # non-end-to-end system
Audio ---------------------------------------> Transcript    # end-to-end deep learning system
```

  - Face recognition system:

```
Image ----------------------> Face recognition    # end-to-end deep learning system
Image --> Face detection --> Face recognition    # deep learning system - best approach for now
```

  - Machine translation system:

```
English --> Text analysis --> ... --> French    # non-end-to-end system
English ----------------------------> French    # end-to-end deep learning system - best approach
```

  - Estimating child's age from the x-ray picture of a hand:

```
Image --> Bones --> Age    # non-end-to-end system - best approach for now
Image ------------> Age    # end-to-end system
```

# Whether to use end-to-end deep learning



- Pros of end-to-end deep learning:
  - Let the data speak. By having a pure machine learning approach, your NN learning input from X to Y may be more able to capture whatever statistics are in the data, rather than being forced to reflect human preconceptions.
  - Less hand-designing of components needed.
- Cons of end-to-end deep learning:
  - May need a large amount of data.
  - Excludes potentially useful hand-design components (it helps more on the smaller dataset).
- Applying end-to-end deep learning:
  - Key question: Do you have sufficient data to learn a function of the **complexity** needed to map x to y?
  - Use ML/DL to learn some individual components.
  - When applying supervised learning you should carefully choose what types of X to Y mappings you want to learn depending on what task you can get data for.

FSB